Running the Gemini ECI Estimation: Step-by-Step Guide

Oct 22, 2026 · @Someone

What this does and what you need

The full run asks Gemini to estimate the A–C embodied carbon range of each of the 292 buildings, 10 times each: 2,920 requests in total. They are sent to Google in one go as a batch job, which Google works through on its own servers, usually within minutes to a few hours. Each building gets its own results file, and a second script then compares the estimates against the ground truth.

Everything lives in one folder on your Desktop: `CEE226-reference`. Before you start, check that it contains these items. Open Finder, go to Desktop, then open `CEE226-reference`.

Item	Where it goes	What it is
`gemini_lca_batch.py`	`CEE226-reference`	The script that sends the requests to Gemini and collects the answers
`LLM-data-cleaning.py`	`CEE226-reference`	The script that builds the results table
`make_fig1_bands.py`	`CEE226-reference`	The script that draws the band figure and counts the misses (Step 5)
`buildings_metadata_LLM.xlsx`	`CEE226-reference/references`	The 292 buildings, one row each
`ground_truth.csv`	`CEE226-reference`	The true ECI for each building
`.env`	`CEE226-reference`	Holds your Gemini API key (see below)
`data_glossary_LLM.xlsx`	`CEE226-reference/references`	Feature descriptions given to the model

Item	Where it goes	What it is
`250524_CLF-LCA-Practice-Guide.txt`	`CEE226-reference/references`	LCA guide given to the model
`1.pdf` to `5.pdf`	`CEE226-reference/references`	Background papers given to the model

The API key. The `.env` file is a plain text file with one line: `GEMINI_API_KEY=` followed by your key, with no spaces or quotes. Its name starts with a dot, so Finder hides it. Press Cmd + Shift + . (period) in Finder to show hidden files.

Other requirements. The laptop needs to be on and online only while you submit the run (Step 1) and collect the results (Step 2). In between, the job runs on Google's side, so you can close the lid or shut down.

One-time setup: install the Python packages

The scripts need seven add-on packages. Install them once per computer (or once per environment, see below); you don't need to repeat this before each run.

1. Open Terminal (Cmd + Space, type `Terminal`, press Return).
2. Paste this and press Return:

   ```
   python -m pip install google-genai pandas openpyxl PyPDF2 python-dotenv matplotlib scipy
   ```

   Starting with `python -m pip` makes sure the packages go into the same Python that runs the scripts. Installing takes a minute or two and prints many lines; it is done when the prompt comes back and the last lines start with `Successfully installed` or `Requirement already satisfied`.
3. Check that everything is in place. Paste this and press Return:

   ```
   python -c "import google.genai, pandas, openpyxl, PyPDF2, dotenv, matplotlib, scipy; print('All packages OK')"
   ```

   You should see `All packages OK`. NumPy warnings above it are harmless (see Troubleshooting).

Package	Used for
`google-genai`	Talking to Gemini
`pandas`	Reading the Excel and CSV files; also installs NumPy, which the results script uses
`openpyxl`	Lets pandas open `.xlsx` files
`PyPDF2`	Reading the text of the background PDFs
`python-dotenv`	Reading your API key from the `.env` file
`matplotlib`	Drawing the band figure (Step 5)
`scipy`	The 95% confidence interval for the share of buildings inside the band (Step 5)

Using a separate `gemini` environment (recommended, especially when sharing with other students). A clean conda environment keeps these packages apart from the rest of Anaconda and avoids the NumPy warnings caused by older packages in `base`. Create it once, including `matplotlib` and `scipy`:

```
conda create -n gemini python=3.12 -y
conda activate gemini
python -m pip install google-genai pandas openpyxl PyPDF2 python-dotenv
matplotlib scipy
```

After that, run `conda activate gemini` each time you open Terminal, before any command in this guide. The prompt then starts with `(gemini)` instead of `(base)`. If a script says `No module named 'matplotlib'` or `'scipy'`, you are in an environment where they are not installed: activate `gemini`, or run the install command again in the current one.

No other packages are needed. `requests`, which was in an earlier version of this command, is no longer used by the scripts; having it installed does no harm.

If a script says `ModuleNotFoundError: No module named …`, one of these packages is missing from the Python that ran it. Run the install command in step 2 again. If the error persists, you may have more than one Python on the Mac: run `which python`, and if the result does not start with `/opt/anaconda3`, send it to Claude.

Before the full run: check the settings

The settings are plain lines near the top of `gemini_lca_batch.py`. For a normal run, leave them as they are. The run name is not a setting: you type it with every command (Step 1).

First, use the latest script. Replace your `gemini_lca_batch.py` with the version Claude shared alongside this guide. It hides two harmless pandas warnings and shows how long each job took.

How to edit the file safely. Open it in PyCharm (right-click the file → Open With → PyCharm). Do not use TextEdit: it can turn straight quotes `"` into curly quotes `“`, which breaks the script. Change only the value after the `=` sign, keep the spacing and quotes as they are, then save with Cmd + S.

Setting	Value for the full run	What it controls
`GEMINI_MODEL`	`"gemini-3.8-flash"`	Which Gemini model is used
`THINKING_LEVEL`	`"medium"`	How long the model reasons per answer
`TEMPERATURE`	`1.0`	Randomness between the 10 runs
`RETRY_FAILED`	`False`	Redo failed runs (used in Step 3 only)
`MAX_OUTPUT_TOKENS`	`16384`	Longest answer allowed, including the model's thinking
`MAX_FILE_MB`	`1000`	Largest request file; bigger runs are split into several files (Google's limit is 2 GB)

Words such as `None`, `True` and `False` start with a capital letter and have no quotes. Text values such as the model name keep their quotes.

The prompts. The text sent to the model is in the section of the script marked `Prompt - edit here`: `build_cached_prefix` holds the part shared by every building, and `build_building_prompt` the part that changes per building. The two lines about the background papers (`1.pdf` to `5.pdf`) are commented out with `#`. Remove the `#` from both lines to include the papers, and put it back on both to leave them out; never on just one of the two.

Step 1: Start the full run

One command sends all 2,920 requests to Google as a batch job. A command is typed (or pasted) into Terminal and run by pressing Return.

1. Open Terminal: press Cmd + Space, type `Terminal`, press Return.
2. Move into the project folder. Paste this and press Return:

   ```
   cd ~/Desktop/CEE226-reference
   ```

   The prompt should now end in `CEE226-reference %`. That tells you which folder Terminal is in.
3. **Choose a run name and write it down.** It becomes the results folder, `results/<run name>/`, and you type it with every later command. Use the date, the hour (24-hour clock) and a short description of what is different about this run, with no spaces, for example `20261022_15_no-background`. Check the year: a typo such as `2025` sorts the folder with last year's runs.
4. Submit the run. Paste this, with your run name, and press Return:

   ```
   python gemini_lca_batch.py submit --run-name 20261022_15_no-background
   ```

   No `--projects` or `--runs` means all 292 buildings, 10 runs each.
5. Wait until the prompt (`CEE226-reference %`) comes back, and don't press Ctrl + C while it says `Uploading`. Without the background papers this takes under a minute; with them, the files are about 1 GB and the upload can take several minutes.

What you will see.

```
2920 requests in 1 file(s); about 19M input tokens (rough estimate).
Model=gemini-3.8-flash, thinking=level=medium
Uploading requests_20261022_150023_part1.jsonl (77 MB) ...
  started batches/f6nqpixhq7ga5227x8zio2vf39cs5s8sjow9
Job list saved to /Users/ehjung/Desktop/CEE226-
reference/results/20261022_15_no-background/batch/jobs.json
```

- `2920 requests` confirms the size of the job: 292 buildings × 10 runs. A smaller number means that folder already holds some runs, which are skipped.
- `input tokens` shows how big the prompts are: about 19M without the background papers, about 250M with them. If the number is not what you expect, press Ctrl + C before the `started` line appears and check the prompt section of the script.

- `2 file(s)` appears when the background papers are included: the requests are larger than `MAX_FILE_MB`, so they are split into two files, each sent as its own job. You then see `Uploading` and `started` twice. Step 2 combines the results.
- `Job list saved to …` means it worked. The job is now with Google.
- Any line starting with `Warning:` and ending in `not found` means an input file is missing. Press Ctrl + C, fix it, and submit again: without that file the model works without its background material.

While it runs: the laptop can be off

Once Step 1 has printed `Job list saved to …`, the job runs entirely on Google's servers. You can close the lid, shut down or go offline; `caffeinate` and overnight logs are no longer needed. The laptop only has to be on and online for submitting (Step 1) and collecting (Step 2).

Collect within about a day of the job finishing. Google keeps the results only for a limited time, and nothing is in your folder until you collect.

Hands-off option. If you'd rather leave the laptop on and have the results collected for you, use this instead of the commands in Step 2:

```
caffeinate -i python gemini_lca_batch.py wait --run-name 20261022_15_no-background
```

It checks every 5 minutes and collects the results as soon as the job is done. Leave Terminal open; it ends with `All jobs collected`.

Step 2: Watch the progress and know when it is done

Check the job whenever you like, from the project folder (`cd ~/Desktop/CEE226-reference` first), with your run name:

```
python gemini_lca_batch.py status --run-name 20261022_15_no-background
```

It prints one line per request file, for example `requests_20261022_150023_part1.jsonl: running`. The word after the colon tells you where the job is:

Status	Meaning
`pending` or `queued`	Waiting for Google to start it
`running`	Being processed
`succeeded`	Finished; ready to collect
`partially_succeeded`	Finished, but some requests failed; collect it, then see Step 3
`failed`, `cancelled` or `expired`	Ended without results; see Troubleshooting
`collected`	Already written into your results folder

When it has finished, the line also shows how long it took:

```
requests_20261022_150023_part1.jsonl: succeeded  total 14 min (queued 3 min)
```

`total` runs from submitting to finishing; `queued` is the wait before Google started. A full run usually takes minutes to a few hours. Google promises it within 24 hours.

Collect the results once every file shows `succeeded`:

```
python gemini_lca_batch.py collect --run-name 20261022_15_no-background
```

It takes a few seconds and ends like this:

```
requests_20261022_150023_part1.jsonl: 2920 results written, 6 failed to parse
All jobs collected. Results: /Users/ehjung/Desktop/CEE226-
reference/results/20261022_15_no-background
```

Note the number of failed runs; with two files, add the two numbers. If it is 0, skip Step 3. If `collect` says a file is still running, nothing is lost: run it again later.

Where the results are. To open the results folder in Finder, run `open results` in Terminal (from `CEE226-reference`). Inside the run's folder:

- `project-1_runs.csv` to `project-292_runs.csv`: one file per building, one row per run.
- `raw/`: the model's full written answer for every run, for example `project-41_run3.md`. Useful when checking why a run failed.

- `batch/` : the job's paperwork. `jobs.json` is the job list that `status` and `collect` read, so keep it. The `requests_*.jsonl` files are what was sent and can be large (about 1 GB with the background papers); you can delete them after collecting.

The CSV files are written all at once by `collect` ; until then the folder holds only `batch/` . You can open them in Excel to look, but do not save them from Excel, or later steps may overwrite your changes or misread the file.

If the run stops: resume it

Nothing on your laptop can stop a submitted job: it runs on Google's side. What can be interrupted, by Ctrl + C, a lost Wi-Fi connection or a closed Terminal, is the submitting or the collecting. Nothing already finished is lost.

- **`collect` was interrupted.** Run the same `collect` command again. It picks up every file not yet collected.
- **`submit` was interrupted before any `started batches/…` line.** Nothing was sent. Run the same `submit` command again.
- **`submit` was interrupted after a `started` line** (only possible with two files). The first job is running. Run `status` to see it, collect it when it has finished, then run the same `submit` command again: it sends only the runs that are still missing.

Why the run name matters. Everything for a run lives in `results/<run name>/` , including the job list. Type the same run name for `submit` , `status` and `collect` . A different name creates a new, empty folder, and `submit` sends all 2,920 requests again, which is billed again. If you are not sure whether a job is already running, run `status` with each name you might have used before submitting.

To cancel a submitted job on purpose, paste this with the job's name, the `batches/…` text from the `started` line in Step 1 (it is also in `batch/jobs.json`):

```
python -c "from google import genai; from dotenv import load_dotenv;
load_dotenv();
genai.Client().batches.cancel(name='batches/f6nqpixhq7ga5227x8zio2vf39cs5s8sj
ow9')"
```

Step 3: Redo failed runs

A few failed runs out of 2,920 are normal. Redo them once the full run has finished, so every building has 10 usable estimates before you analyse the results.

What a failed run is. A run counts as failed when the script could not get a usable range from it. In that building's CSV, the row has `parse_ok` = `False` and the `failure_type` column says why:

`failure_type`	What happened	What usually fixes it
`api_error`	Google returned an error for this request instead of an answer	Redo it; it almost always works the second time
`no_range`	The model answered but left out the final `RANGE_kgCO2e_per_m2:` line	Redo it; the next answer usually includes the line
`truncated`	The answer was cut off because it ran out of output tokens	Redo it; if it keeps happening, raise `MAX_OUTPUT_TOKENS`
`invalid_range`	The range made no sense (low above high, or zero or below)	Redo it, then read the raw answer if it repeats

How to redo them.

1. Open `gemini_lca_batch.py` in PyCharm and change `RETRY_FAILED = False` to `RETRY_FAILED = True`. Save with Cmd + S.
2. In Terminal, submit again with your run's name:

   ```
   cd ~/Desktop/CEE226-reference
   python gemini_lca_batch.py submit --run-name 20261022_15_no-background
   ```

   It sends only the failed runs, for example `6 requests in 1 file(s)`.
3. Check it with `status`, and when it shows `succeeded`, run `collect` with the same name, as in Step 2. The failed rows are replaced in place. A job this small usually finishes within minutes.
4. Check the `collect` line. If it says `0 failed to parse`, you are done. If the number is above 0, repeat steps 2 and 3.
5. **Change `RETRY_FAILED` back to `False`** and save. If you leave it on, a future submit also resends failed runs, which is harmless but easy to forget.

If the same building keeps failing. After two or three retries, look at its raw answer in `results/<your run name>/raw/`, for example `project-87_run4.md`. Open it with TextEdit or PyCharm (reading only). If the answer looks reasonable but the final line is missing or formatted differently, note the building and send me the file. An `api_error` row has no raw file, because no answer came back. Repeated `api_error` rows usually mean an outage on Google's side: wait an hour and submit again.

A building can be reported with fewer than 10 usable runs. When you run Step 4, Terminal lists such buildings, for example `project-87 9/10`, so you can mention them in your methods.

Step 4: Create the final summary file

The last step runs `LLM-data-cleaning.py`. It reads every `project-#_runs.csv` from your run's folder, compares each building's estimate with `ground_truth.csv`, and saves one summary CSV. That summary is the file you analyse and report.

Before you start, check:

- The results have been collected (`collect` printed `All jobs collected`) and failed runs have been redone (Step 3).
- `LLM-data-cleaning.py` and `ground_truth.csv` are both directly inside `CEE226-reference`, next to `gemini_lca_batch.py`.

How to run it.

1. Open Terminal (Cmd + Space, type `Terminal`, press Return).
2. Go to the project folder. Paste and press Return:

   ```
   cd ~/Desktop/CEE226-reference
   ```

3. Run the script with your run's folder name, the one you wrote down in Step 1:

   ```
   python LLM-data-cleaning.py 20261022_15_no-background
   ```

   Replace `20261022_15_no-background` with your own folder name. You can also leave the name out (`python LLM-data-cleaning.py`); the script then uses the newest run folder. That is only safe if you have not started another run since.
4. It takes a few seconds. Wait until the prompt (`CEE226-reference %`) comes back.

What you will see in Terminal. Ignore any NumPy warnings at the top. The useful lines look like this:

```
Ground truth: /Users/ehjung/Desktop/CEE226-reference/ground_truth.csv (292
projects)

Projects: 292   tests per project: 10

RMSE                 : … kgCO2e/m2
Mean absolute error  : … kgCO2e/m2
Mean ratio est/true  : …
SD within project    : …
SD across projects   : truth … | estimate …
Range across projects: truth … | estimate …

Saved: /Users/ehjung/Desktop/CEE226-reference/results/20261022_15_no-
background/20261022_1640_results_project-292_10.csv
```

The per-building table is printed in Terminal only for 40 buildings or fewer. For the full 292, it is in the saved file only.

Lines that start with `!` are warnings. Read them before using the file.

- `! projects with failed runs (parsed/total): project-87 9/10, …` means those buildings' averages use fewer than 10 runs. Go back to Step 3, or note them in your methods.
- `! no usable run data for: […]` means those buildings have no usable runs. They are left out of every summary number.
- `! runs without ground truth (excluded): […]` means a results file has no matching row in `ground_truth.csv`. With your current files this should not appear.

Where the summary file is and how to open it. It is saved inside the run's folder, next to the `project-#_runs.csv` files. To open that folder in Finder, run (with your folder name):

```
open results/20261022_15_no-background
```

Then double-click the file whose name contains `_results_`; it opens in Excel. Its name is the date and time it was made, then `results`, then the last building and the number of runs per building, for example `20261022_1640_results_project-292_10.csv`. If the last number is `_9` or lower, many buildings are missing runs: go back to Step 3.

What is in the file. It has the same layout as your earlier `20260825_1248_results_project-11_10.csv`: one row per building, a blank row, then the summary rows.

Column	Meaning
`Project Name`	The building, for example `project-4`
`ECI_true_kgCO2e_m2`	The true value from `ground_truth.csv`
`LLM_mean_mid_kgCO2e_m2`	The average of the 10 range midpoints for that building
`LLM_sd_mid`	How much the 10 midpoints vary for that building
`abs_error`	Distance between estimate and truth, in $kgCO_2e/m^2$
`ratio_est_over_true`	Estimate divided by truth; 1.00 is exact, above 1 is an overestimate

Summary row	What it tells you
MEAN, MEDIAN	The typical value of each column across all buildings
MIN, MAX, RANGE	The lowest, highest and spread of each column
SD (across projects)	How much buildings differ from each other. An estimate SD much smaller than the truth SD means the model gives similar answers for very different buildings
RMSE (truth vs estimate)	The overall error in $kgCO_2e/m^2$, weighting large misses more heavily

Good to know.

- Running the script again never overwrites an earlier summary. Each run makes a new file stamped with the current date and time, so use the newest one.
- You can run it while the main run is still going, as a progress check. It summarises only the buildings finished so far.

- Use `ground_truth.csv` only with run folders made from `buildings_metadata_LLM.xlsx`. Your older 11-building comparison used the same names (`project-1` to `project-11`) for different buildings, and mixing them gives wrong errors without any warning.
- The true ECI ranges from about 13.5 to 2,160 $kgCO_2e/m^2$. Check the few extreme buildings before reporting RMSE, since it is sensitive to large misses; MEDIAN and the ratio rows are less affected.
- Two settings at the top of `LLM-data-cleaning.py` rarely need changing: `GROUND_TRUTH_PATH` (where the ground truth file is) and `ONLY_PROJECTS_WITH_RUNS` (`True` lists only buildings with results).

Step 5: Plot the bands and count the misses

`make_fig1_bands.py` reads the same `project-#_runs.csv` files and `ground_truth.csv` as Step 4. For each building it draws a band of ±20% around the model's estimate and checks whether the true ECI falls inside it. It makes a figure and a summary file.

Before you start, check:

- Step 4 is done, or at least the results have been collected (Step 2) and failed runs redone (Step 3).
- `make_fig1_bands.py` is directly inside `CEE226-reference`, next to `LLM-data-cleaning.py`.
- `matplotlib` and `scipy` are installed (see One-time setup). If you use the `gemini` environment, run `conda activate gemini` first.

How to run it.

1. Go to the project folder. Paste and press Return:

   ```
   cd ~/Desktop/CEE226-reference
   ```

2. Run the script with your run's name:

   ```
   python make_fig1_bands.py 20261022_15_no-background
   ```

3. It takes a few seconds. Wait until the prompt (`CEE226-reference %`) comes back.

What you will see in Terminal.

```
292 buildings. Saved: /Users/ehjung/Desktop/CEE226-
reference/results/20261022_15_no-background/20261022_eci-llm-values.csv
Saved: /Users/ehjung/Desktop/CEE226-reference/results/20261022_15_no-
background/figures/fig1_bands_sorted.png

Inside: 100/292 = 34.2%   Wilson 95% CI [29.0%, 39.9%]
ABOVE:  171/292 = 58.6%
BELOW:  21/292 = 7.2%

Saved: /Users/ehjung/Desktop/CEE226-reference/results/20261022_15_no-
background/figures/fig1_bands_summary.txt
```

Line	Meaning
`Inside`	Buildings whose true ECI is within ±20% of the estimate, with a 95% confidence interval for that share
`ABOVE`	True ECI above the band: the model underestimated
`BELOW`	True ECI below the band: the model overestimated

A line starting with `! left out` names buildings with no usable runs or no ground truth. They are not in the figure or the counts.

The three files it makes, all inside the run's folder:

File	What it is
`figures/fig1_bands_sorted.png`	The band figure
`figures/fig1_bands_summary.txt`	The three totals, the full ABOVE and BELOW project lists, and a per-building table: truth, estimate, band edges, estimate/truth and the result
`<date>_eci-llm-values.csv`	One row per building: `project_id`, `truth`, `f_llm` (mean of the run estimates). Also the input for the full evaluation script, `20260825_eci-llm-eval.py --data …`

Reading the figure. Each row is one building, sorted from lowest true ECI (bottom) to highest (top). The dot is the true ECI, the tick is the model's estimate, and the bar is the ±20% band. Green means the true value is inside the band; red means it is outside. With more than 40 buildings the rows have no labels and the ECI axis uses a log scale, because the true values span from about 13.5 to 2,160 $kgCO_2e/m^2$. To find a particular building, use the summary file.

Good to know.

- Running it again replaces the figure and the summary file in that run's folder. Copy them elsewhere first if you want to keep an earlier version.
- The ±20% band is a fixed tolerance, not an uncertainty range from the model. Report the share inside it as a descriptive number.
- Three settings at the top of the script rarely need changing: `DELTA` (band half-width, `0.20` = ±20%), `LABEL_MAX_N` (up to this many buildings, each row is labelled) and `LOG_X` (`"auto"`, `True` or `False`).

Troubleshooting

Most messages fall into one of the cases below. If you see something not listed, copy everything from the command you typed down to the end and paste it to Claude.

Message in Terminal	What it means	What to do
`A module that was compiled using NumPy 1.x cannot be run in NumPy 2.2.6`	Some optional add-ons in your Python setup are older than NumPy. Pandas works without them	Ignore it. To remove it for good: `pip install "numpy<2"`
`FileNotFoundError: … buildings_metadata_LLM.xlsx`	The building file is not in `CEE226-reference/references`	Move it there, or check the spelling of the file name
`FileNotFoundError: Ground truth not found`	`ground_truth.csv` is not next to the scripts	Move it into `CEE226-reference`
`Warning: PDF not found` or `Warning: could not read`	A file in `references/` is missing or named differently	Press Ctrl + C before the upload finishes, fix the file, submit again

Message in Terminal	What it means	What to do
`Set GEMINI_API_KEY`	The script cannot find your API key	Check the `.env` file (see the first section)
`can't open file … No such file or directory`	Terminal is in the wrong folder	Run `cd ~/Desktop/CEE226-reference` first
`… job(s) in this run are not collected yet`	That run name already has a submitted job that has not been collected	Run `status`, then `collect`; for a new run, use a new run name
`No batch jobs recorded for run …`	No job was submitted under that run name	Check the spelling of the run name, or run `submit` first
`No run folders with project-*_runs.csv`	`LLM-data-cleaning.py` cannot find results	Give the folder name after the command, as in Step 4
`zsh: command not found: python`	Terminal is not using your Anaconda Python	Use `python3` instead of `python`, or reopen Terminal
`ended as JOB_STATE_FAILED` (or EXPIRED, CANCELLED)	The job ended without results	Run the same `submit` command again; it resends only the missing runs

If the Mac went to sleep or lost power. A submitted job is not affected. If it happened while submitting or collecting, see "If the run stops: resume it" above.

Quick reference

Every command, in the order you use them. Replace `20261022_15_no-background` with your run's folder name.

```
# One-time setup: create the gemini environment with all packages (once per computer)
conda create -n gemini python=3.12 -y
conda activate gemini
python -m pip install google-genai pandas openpyxl PyPDF2 python-dotenv
```

```
matplotlib scipy

# Each time you open Terminal: activate the environment, go to the project
folder
conda activate gemini
cd ~/Desktop/CEE226-reference

# Step 1: submit the full run (all 292 buildings, 10 runs each)
python gemini_lca_batch.py submit --run-name 20261022_15_no-background

# Step 2: check the job, then collect the results once it says succeeded
python gemini_lca_batch.py status --run-name 20261022_15_no-background
python gemini_lca_batch.py collect --run-name 20261022_15_no-background

# Or check every 5 minutes and collect automatically (keeps the Mac awake)
caffeinate -i python gemini_lca_batch.py wait --run-name 20261022_15_no-
background

# Step 3: redo failed runs (set RETRY_FAILED = True first, back to False
after)
python gemini_lca_batch.py submit --run-name 20261022_15_no-background
python gemini_lca_batch.py collect --run-name 20261022_15_no-background

# Step 4: create the final summary file
python LLM-data-cleaning.py 20261022_15_no-background

# Step 5: band figure and inside/above/below counts
python make_fig1_bands.py 20261022_15_no-background

# Open the results folder in Finder
open results
```

Smaller runs, for tests. Each test needs its own run name, and is then checked and collected like a full run:

What you want	Command
One building, 10 runs	`python gemini_lca_batch.py submit --projects 4 --run-name test_p4`
First 10 buildings	`python gemini_lca_batch.py submit --projects 1-10 --run-name test_p1-10`

What you want	Command
All buildings, 4 runs each	`python gemini_lca_batch.py submit --runs 4 --run-name test_runs4`
Write the request files without sending anything	add `--dry-run` to any of the above

Ctrl + C stops only the command running in Terminal (`submit`, `status`, `collect` or `wait`). A submitted job keeps running on Google's side; to cancel it, see "If the run stops: resume it".